# Shapdapters (Shape Adapters).

Brian Linton
brialint@pdx.edu
Rust Programming: Emergent Complexity Course Project.

I want to create a generative art program (I got some inspiration from the Diffusion-Limited Aggregation example you posted) that I'm calling Shapdapters. I hope it will generate some complex and aesthetically pleasing results (and if it doesn't look good I will plead the case that I'm not an art student). The program will be a modular system that will obey the following rules:

1. The system **must** begin with a module that generates shapes and end with a module that cumulatively and visually assembles the shapes together in some manner.
2. In between the bookended generators are 0 or more "shapdapters" that must perform exactly one atomic manipulation to the shape.
3. Each stage of the chain will work on the image produced by the previous stage.

# Minimum Viable Product (MVP) System

The following is what I will implement for this project. It is only one of any number of valid "Shapdapter" chains.

1. The program will take in an argument for the number of rounds to run and an argument for a configuration file containing a list of "Shapdapter" crates to load and use.
2. Generator: Generates a random shape based on a 16x16 square grid (so depending on the shape it might not be exactly 16x16).
    a. Squares
    b. Circles
    c. Horizontal rectangles
    d. Vertical rectangles
3. Shapdatper 1: Scale the image 2x.
4. Shapdapter 2: Remove a random 25% of the image.
5. Shapdapter 3: Color the shape with a rainbow color (ROYGBIV).
6. Assembler.
    a. Attach the shape to the combined shape at a random spot on the perimeter.
    b. It must be completely adjacent to a previously placed shape (directly above, below or to the left or right of it - not at a corner).
    c. Vertical rectangles start from the right side of the "tile" and horizontal rectangles start at the bottom of the "tile"
    d. Fill in any enclosed area of the current shape with another random rainbow color.
7. Output: The program will output a .png file of the result.

# Visual Example:

The following is a visual example of five rounds of the above system. I used random.org's random number generator (I fudged one random number to better illustrate the system) along with photoshop and aesprite to do a manual run:

| Round | Generator | Shapdapter (Scaler) | Shapdapter (Remover) | Shapdapter (Painter) | Assembler (Current image) |
|---|---|---|---|---|---|
| 1 |  |  |  |  |  |
| 2 |  |  |  |  |  |
| 3 |  |  |  |  |  |
| 4 |  |  |  |  |  |
| 5 |  |  |  |  |  |

# Theme Consistency

I believe this fits the emergent complexity theme, first in complex art generation from simple rules but also what can be done with a variety of generator/assembler combinations along with varied numbers of diverse "Shapdapters."

# AI Disclosure

I used ChatGPT to:

1. Do quick research to make sure I actually came up with this idea and it wasn't something I heard of somewhere at some point in time.
2. Do quick research to check that the name wasn't trademarked or used in something super popular and that it wasn't offensive in any way.
3. To check that the images generated in this document weren't some odd symbol someone might find offensive.
4. To make sure I could dynamically load crates at runtime from a config file in rust (I did not ask for code to do this, just verification).